Tactical Football Player Clustering

October 7, 2026

1 Introduction

In modern football, the traditional position of a player rarely tells us what a player's actual tactical role is during a game. For example, a fullback's role in some teams might be very defensive, while in other systems fullbacks might push so far forward that they end up on the wing, much like Quincy Promes. The goal of this project is to identify modern tactical roles and divide players into those categories. This will be done using machine learning, specifically clustering, to group players into clusters based on their in-game statistics, such as expected goals and tackles.

Section 2 discusses the clustering problem and introduces the dataset. Section 3 then describes the data preprocessing, feature selection and the machine learning models and their evaluation. Section 4 presents the results and Section 5 summarizes the findings, limitations and further improvements.

2 Problem Formulation

This application is an unsupervised machine learning clustering problem. The dataset used for the project is Football Players Stats (2024-2025) which is sourced from FBref and published on Kaggle [2]. It has player statistics from the top 5 European leagues (English Premier League, La Liga, Serie A, Bundesliga and Ligue 1) from the 2024-2025 season. It consists of 2854 data points before processing, each representing a player. Each data point has 267 columns, which contain player information and in-game statistics. For this project, only 30 of these statistics are used. The statistics are chosen by hand, so that every category is included equally to avoid a single category dominating the output. Five features are used for each category: finishing, passing, defending, creativity, dribbling, and positioning and duels. All features are continuous and numeric. A complete list and description of all selected features is provided in Appendix A, Table 2. Since an unsupervised method is used, there are no labels. We are trying to discover the roles, which will be the labels of the clusters. After hand-picking 30 features, K-Means is used to cluster players, and finally the clusters are named to fit the players' playing styles.

3 Methods

3.1 Dataset and Features

The dataset has over 260 columns. However, only a fraction of these will be used because many of the columns overlap each other, telling the same or very similar information. Thirty suitable features are selected by hand. The columns have the statistics of the players for the whole season, but for this problem, statistics per 90 minutes are needed so that playing time will not affect the result. Players with little playing time are also filtered out, 900 minutes of playing time is used for the minimum limit. Goalkeepers are excluded from this problem because field players' roles are considered more interesting and goalkeepers' statistics are not compatible with field player statistics. Every feature is standardized because K-Means is sensitive to scale. PCA is used to reduce the dimensionality. 90 percent of the variance is kept, which results in 10 dimensions for the feature vector. After data processing, 1450 data points remain, which will be given to the model.

3.2 Machine Learning Models and Model Validation

The machine learning models used are K-Means and DBSCAN. For this problem, K-Means is probably the more suitable of the clustering models available because each player should be assigned to exactly

one cluster, and K-Means lets us control the number of clusters through k. DBSCAN groups points by density so it might produce larger clusters because tactical roles can overlap. Also, players in sparse regions might be excluded completely because DBSCAN would treat them as noise. The features of this problem are numeric and their Euclidean distance tells directly how similar they are, so K-Means works well. The result of DBSCAN will tell if the roles exist as dense and separated islands or as an overlapping and continuous space.

The value of k is not decided in advance. K will be selected from the range 2–15 by comparing the elbow of the clustering error against silhouette scores. Silhouette score measures how similar a data point is to its own cluster compared to other clusters. The higher the silhouette score, the better the model. The upper bound of k is 15, so that the clusters are big enough to be interpreted clearly.

A more specific split is done by running K-Means again inside a cluster to refine the roles further. Each main cluster is re-standardized and gets its own PCA before sub-clustering.

K-Means works by minimizing the clustering error which is the average squared Euclidean distance between each data point and the centroid of its cluster [1] :

$$\widehat{L}\left(\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}, \{\hat{y}^{(i)}\}_{i=1}^{m} \mid \mathcal{D}\right) = \frac{1}{m}\sum_{i=1}^{m}\left\|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(\hat{y}^{(i)})}\right\|^2. \tag{1}$$

DBSCAN on the other hand groups two data points into a shared cluster if they are reachable from one another. Reachability between points $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(i')}$ requires a continuous path connecting $\mathbf{x}^{(i')}$ to $\mathbf{x}^{(i)}$. This connecting sequence is made up of points separated by a maximum distance of ε. Additionally, all points along this route must be core points, meaning they contain at least a threshold number of neighbors within the ε radius [1]. The suitable ε is chosen by the elbow method from Figure 7. Here, $\varepsilon = 3.5$ is chosen.

Since this is an unsupervised clustering task and the goal is to find the real roles of the players, there is no prediction of future data. Because of this, no separate validation or test set is held out and all the 1450 filtered players are used to identify the cluster structure. However, the roles could still be applied to new players by assigning each player to the nearest centroid.

4 Results

The models were evaluated using inertia (elbow method) and silhouette scores as mentioned earlier. The initial 30-feature K-Means model had a poor silhouette score so PCA was used to reduce the dimensionality of the features. The performance was still quite poor for over 5 clusters so I decided to use 4 clusters first and then use K-Means again for each cluster to find the sub clusters. With k = 4, the silhouette score is about 0.24 (Figure 1). K-Means struggles to separate the players into over 5 clusters at first, probably because, for example, the stats of a centre-back and a striker differ so much that it dominates the output and the underlying differences do not get noticed.

After the first clustering the standardization and PCA are run again within each main cluster. K is again chosen with the elbow method and silhouette score. This gives 3 sub-clusters for main cluster 0, 2 clusters for main cluster 1, 4 clusters for main cluster 2 and 3 clusters for main cluster 3. For main cluster 2, k = 4 was chosen despite the higher silhouette at k = 2, because the elbow is at 4 and two clusters would merge clearly different midfield roles. Scores for the sub-clusters are in the appendix under Figures 3, 4, 5, 6. The silhouette scores were quite low across the board, only 0.14-0.20. This means that the clusters overlap a lot. I decided that this is still good enough for this type of continuous performance data.

DBSCAN did not work for this data and problem. With $\varepsilon = 3.5$, which was close to optimal, the model only found 1 big cluster. When ε was decreased, the model found more clusters but there were many players that were classified as noise and left out. For example, with ε of 2.2 there were 373 noise

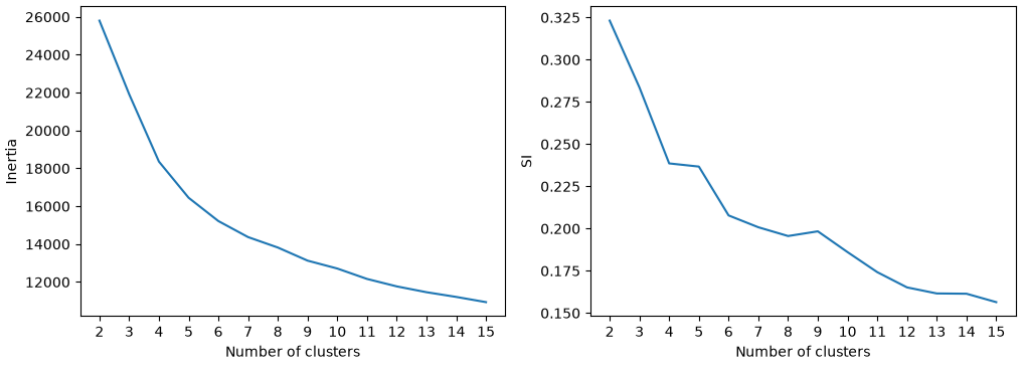


Figure 1: Inertia and Silhouette score for the main clusters

points. DBSCAN struggled with the player data because the data is continuous and there are no big gaps between the clusters. This matches the low silhouette scores.

After the second clustering there are 12 clusters. The weights of the different stats are visualized in Figure 2. Red means that a statistic is above the mean of the 12 clusters and blue means that a statistic is below the mean.

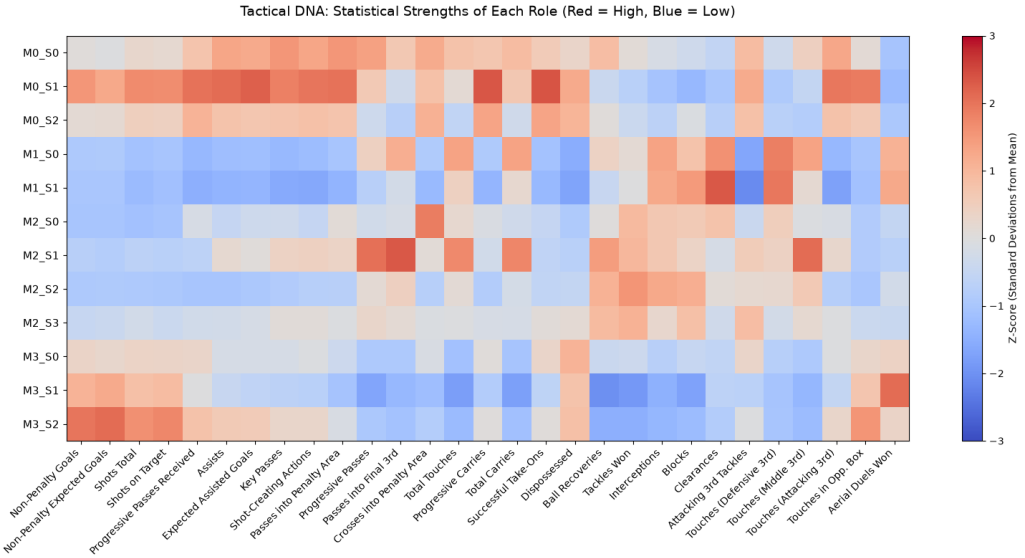


Figure 2: Heatmap of statistical strengths of the different clusters relative to the mean of the 12 roles

5 Conclusion

The chosen method for this problem was K-Means. Even though the silhouette scores were low, there were clear differences between the clusters which allowed them to be named descriptively. DBSCAN on the other hand did not work for this problem because it left too many players out or made just one cluster. The original hypothesis was right. Football positions are fluid and the position of a player does not tell the whole truth of what the player's role is on the field. The result is 12 unique tactical roles. Also, by analyzing the player positions some interesting outliers were found, for example Joško Gvardiol in Deep-Lying Playmakers.

There were also some problems. In this project, only one season is analyzed and the features are hand-picked. Furthermore, the playing style of the team is not considered at all. The playing style of a team affects player statistics which could be reduced by possession-adjusting the statistics. Also, the clustering is affected by how much a certain player produces. So, it also separates good players from less productive players, which affects the results. For example, the main difference between M0_S1

and M0_S2 is that M0_S2 has lower values for almost every statistic which can be interpreted as those players just being worse. However, there were still clear categories. I named the clusters according to Figure 2 and my football knowledge. Each cluster was named by looking at which features are clearly above or below the average of the roles in the heatmap, what is the most common traditional position in that cluster and sample players from the clusters.

Table 1: Player roles with the most common position and example players of the cluster

Code	**Role**	**Position**	**Example players**
M0_S0	Attacking Playmakers	MF	Tijjani Reijnders, Bruno Fernandes, Alex Iwobi, Enzo Fernández, Mikkel Damsgaard
M0_S1	Elite Inside Forwards	FW	Mohamed Salah, Cole Palmer, Kylian Mbappé, Lamine Yamal, Raphinha
M0_S2	Direct Wingers	FW, MF	Bryan Mbeumo, Antoine Semenyo, Morgan Rogers, Dodi Lukebakio, Carlos Vicente
M1_S0	Ball-Playing Centre-Backs	DF	Amir Rrahmani, Virgil van Dijk, Florian Lejeune, Levi Colwill, Gianluca Mancini
M1_S1	Traditional Centre-Backs	DF	Federico Baschirotto, Nathan Collins, Obite N'Dicka, Max Kilman, Dan Burn
M2_S0	Crossing Fullbacks	DF	Milos Kerkez, Giovanni Di Lorenzo, Angeliño, Antonee Robinson, Jackson Tchatchoua
M2_S1	Deep-Lying Playmakers	MF	Joško Gvardiol, Mattéo Guendouzi, Marten de Roon, Federico Valverde, Youri Tielemans
M2_S2	Ball-Winning Midfielders	MF	Moisés Caicedo, Emmanuel Gyasi, Remo Freuler, Jesper Karlström, Ryan Gravenberch
M2_S3	Box-to-Box Players	MF	Bruno Guimarães, Daniel Muñoz, Rayan Aït-Nouri, Andrei Ratiu, Simon Sohm
M3_S0	Box-Crashers	FW, MF	Scott McTominay, Tim Kleindienst, Diego López, Nadir Zortea, Mauro Arambarri
M3_S1	Target Strikers	FW	Roberto Piccoli, Nikola Krstović, Javi Puado, Chris Wood, Ante Budimir
M3_S2	Complete Forwards	FW, MF	Alexander Isak, Ismaila Sarr, Hugo Ekitike, Jonathan David, Raúl Jiménez

A soft clustering method such as Gaussian Mixture Model could have also worked for this problem. It would give each player a degree of belonging in [0,1] to several roles, which is left for future work.

6 Use of AI

In the making of this project and report, Anthropic's Claude and Google's Gemini were used as tools. Specifically, they were used during the early stages for topic ideation and brainstorming. They were also used as a help for Python syntax and visualizations. During the writing process, they were used to proofread the text and fix grammatical errors. Additionally, the AI was used to assist with technical formatting in Overleaf, including generating the LaTeX code required for the tables in this document.

References

[1] A. Jung. *Machine Learning: The Basics*. Springer, Singapore, 2022.

[2] Hubert Sidorowicz. Football players stats (2024-2025), 2025. Accessed: 2026-09-11. URL: `https://www.kaggle.com/datasets/hubertsidorowicz/football-players-stats-2024-2025`.

A Tables

Table 2: Features (per 90 minutes unless a percentage).

Category	Feature	Description
Finishing	G-PK	Non-Penalty Goals
	npxG	Non-Penalty Expected Goals
	Sh	Shots Total
	SoT	Shots on Target
	PrgR	Progressive Passes Received
Creativity	Ast	Assists
	xAG	Expected Assisted Goals
	KP	Key Passes
	SCA	Shot-Creating Actions
	PPA	Passes into Penalty Area
Passing	PrgP	Progressive Passes
	Cmp%	Pass Completion %
	1/3	Passes into Final 3rd
	CrsPA	Crosses into Penalty Area
	Touches	Total Touches
Carrying and dribbling	PrgC	Progressive Carries
	Carries	Total Carries
	Succ	Successful Take-Ons
	Dis	Dispossessed
	Recov	Ball Recoveries
Defending	Tkl	Tackles
	Int	Interceptions
	Blocks	Blocks
	Clr	Clearances
	Tkl Att 3rd	Attacking 3rd Tackles
Positioning and duels	Def 3rd	Touches (Defensive 3rd)
	Mid 3rd	Touches (Middle 3rd)
	Att 3rd	Touches (Attacking 3rd)
	Att Pen	Touches in Opp. Box
	Won	Aerial Duels Won

Table 3: Player information columns. Not used as input to the clustering.

Column	Description	Use
Player	Player name	Identifying players in clusters
Pos	Position listed by FBref	Excluding goalkeepers; comparison with clusters
Squad	Club	Interpreting clusters
Comp	League	Interpreting clusters
Min	Minutes played	Filtering players ($\geq$ 900 minutes)

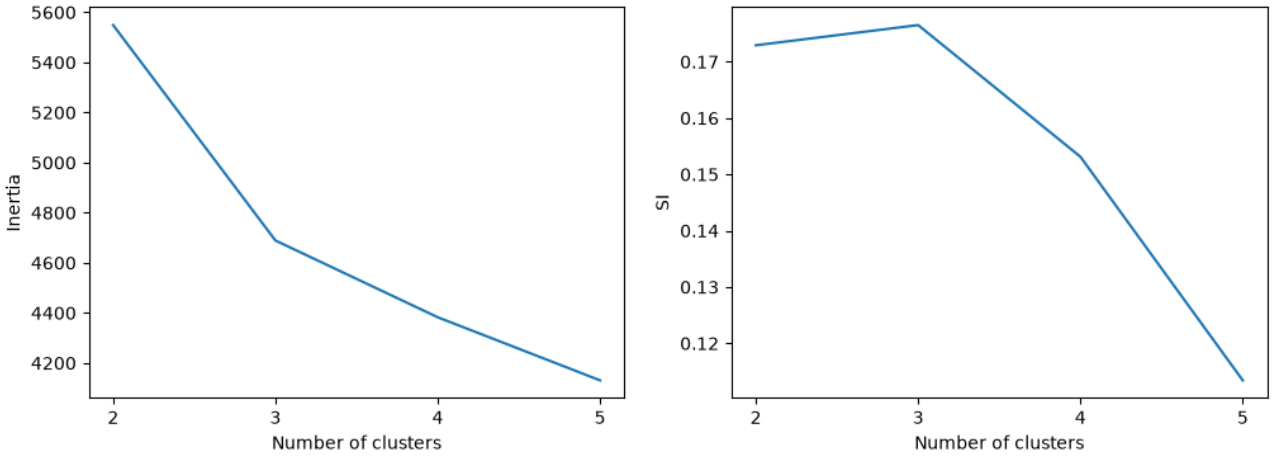


Figure 3: Inertia and Silhouette score for the main cluster 0 sub-clusters

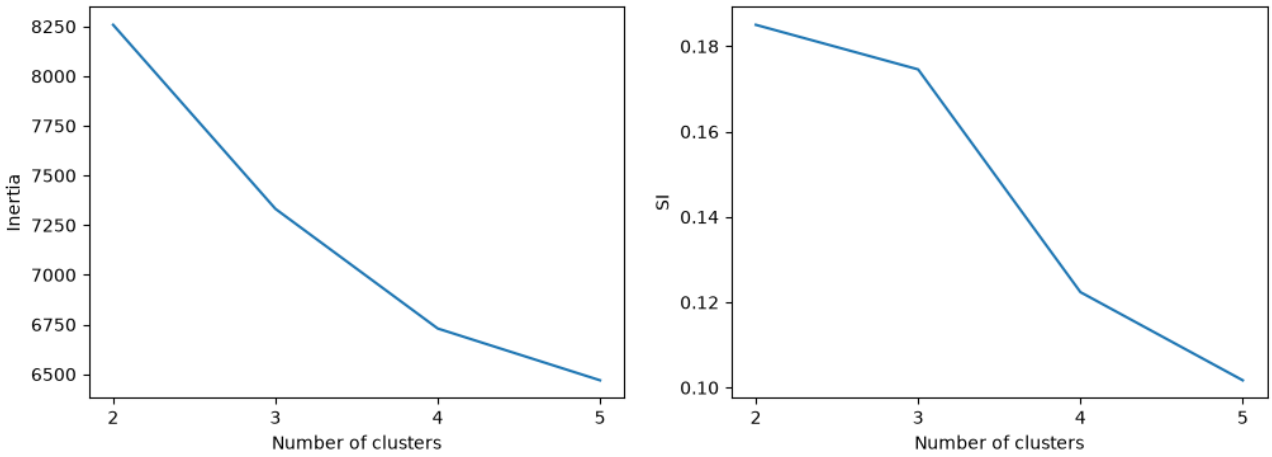


Figure 4: Inertia and Silhouette score for the main cluster 1 sub-clusters

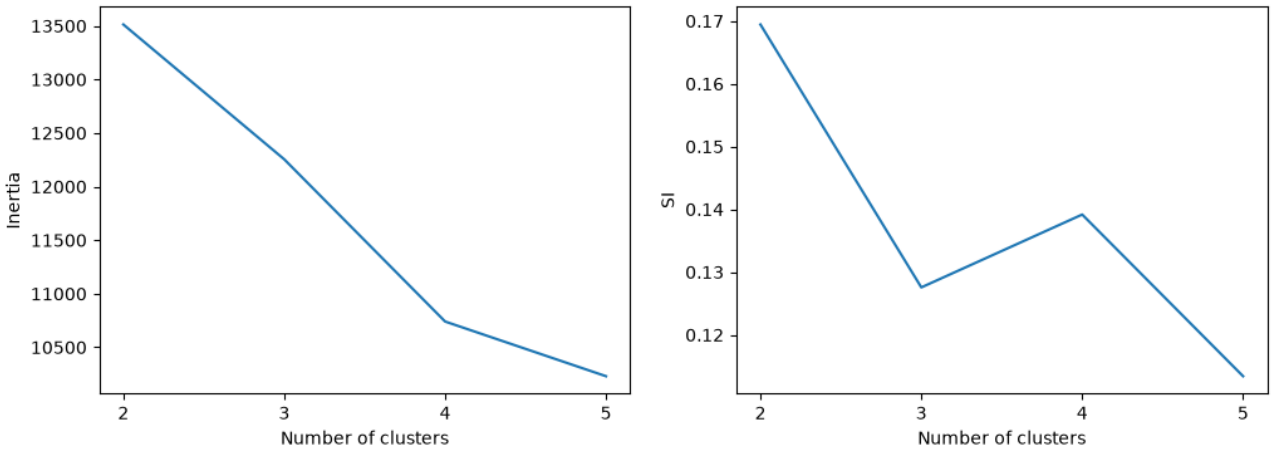


Figure 5: Inertia and Silhouette score for the main cluster 2 subclusters

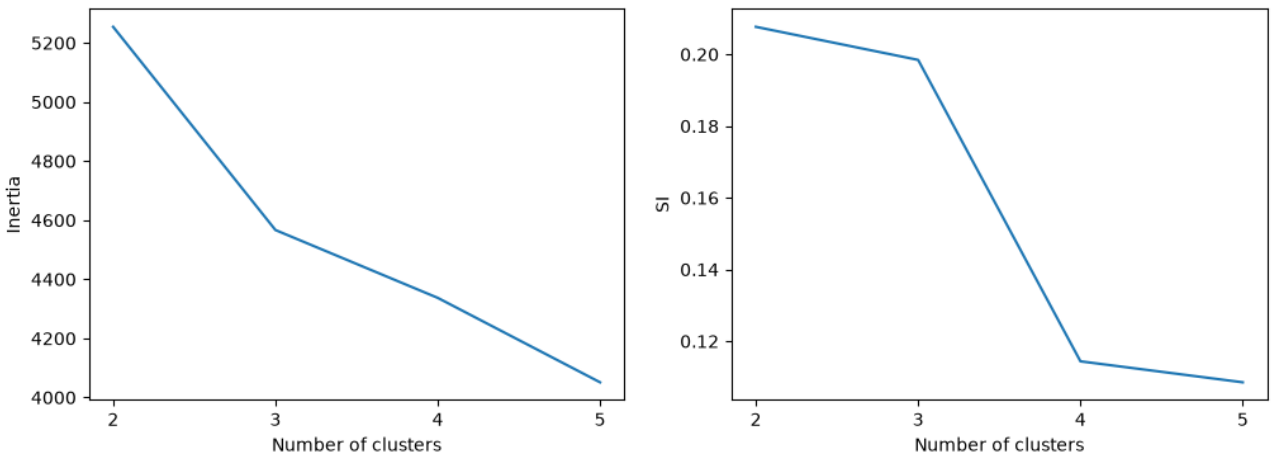


Figure 6: Inertia and Silhouette score for the main cluster 3 subclusters

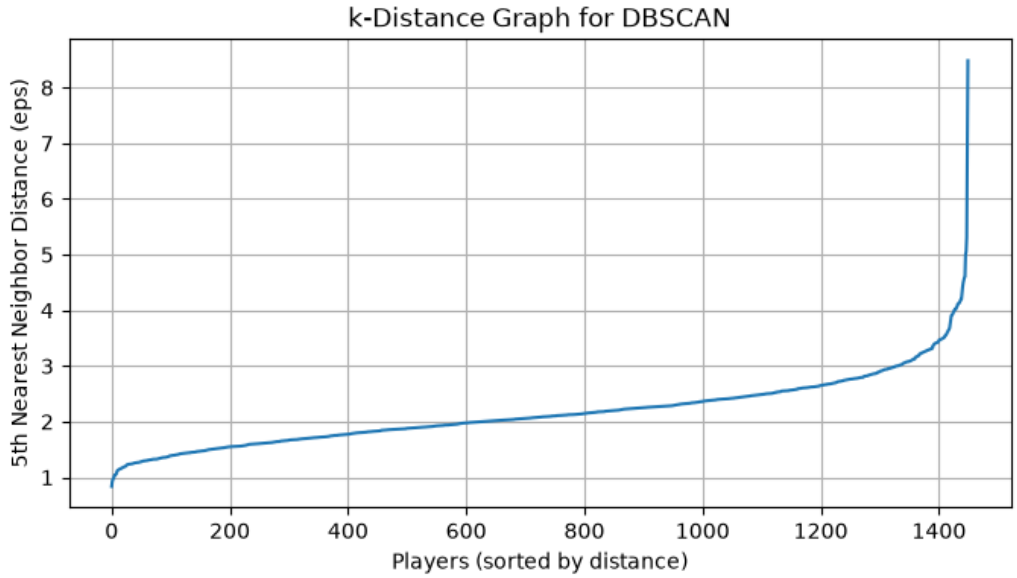


Figure 7: DBSCAN evaluation graph

B Graphs

C Project code

Listing 1: Project Code.

```
1
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import pandas as pd
5
6  from sklearn.cluster import KMeans
7  from sklearn import metrics
8  from sklearn.preprocessing import StandardScaler
9  from sklearn.cluster import DBSCAN
10 from sklearn.neighbors import NearestNeighbors
11 from sklearn.decomposition import PCA
12
13
```

```
14
15 df = pd.read_csv('data/players_data-2024_2025.csv')
16
17 df = df[(df['Min'] >= 900) & (df['Pos'] != 'GK')]
18
19 info_columns = ['Player', 'Pos', 'Squad','Comp', 'Min']
20
21 features = [
22     'G-PK', 'npxG', 'Sh', 'SoT', 'PrgR',
23     'Ast', 'xAG', 'KP', 'SCA', 'PPA',
24     'PrgP', 'Cmp%', '1/3', 'CrsPA', 'Touches',
25     'PrgC', 'Carries', 'Succ', 'Dis', 'Recov',
26     'Tkl', 'Int', 'Blocks_stats_defense', 'Clr', 'Att 3rd',
27     'Def 3rd_stats_possession', 'Mid 3rd_stats_possession',
28     'Att 3rd_stats_possession', 'Att Pen', 'Won',
29 ]
30
31 df_new = df[info_columns + features].copy()
32
33 print(len(df_new), df_new[features].isna().sum().sum())
34
35 numerical_features = [f for f in features if f != 'Cmp%']
36
37 per_90 = df_new[numerical_features].div(df_new['Min'] / 90, axis=0)
38 df_p90 = pd.concat([df_new[info_columns], per_90, df_new[['Cmp%']]], axis=1)
39
40 X = StandardScaler().fit_transform(df_p90[numerical_features + ['Cmp%']])
41
42 pca = PCA(n_components=0.90, random_state=0)
43 X_pca = pca.fit_transform(X)
44
45 print(f"Dimensions reduced from {X.shape[1]} to {X_pca.shape[1]}")
46
47 inertia = []
48 SI = []
49
50 for k in range(2, 16):
51     kmeans = KMeans(random_state=0, n_clusters=k)
52     kmeans.fit(X_pca)
53
54     si = metrics.silhouette_score(X_pca, kmeans.labels_)
55
56     inertia.append(kmeans.inertia_)
57     SI.append(si)
58
59 plt.figure(figsize=(12,4))
60 plt.subplot(1,2,1)
61 plt.plot(np.arange(2,16),inertia)
62 plt.xlabel('Number of clusters')
63 plt.ylabel('Inertia')
64 plt.xticks(np.arange(2,16))
65 plt.subplot(1,2,2)
66 plt.plot(np.arange(2,16),SI)
67 plt.xlabel('Number of clusters')
68 plt.ylabel('SI')
69 plt.xticks(np.arange(2,16))
70 plt.show()
71
72 kmeans_first = KMeans(random_state=0, n_clusters=4)
73 df_p90['Main_Cluster'] = kmeans_first.fit_predict(X_pca)
74
75 for subset_id in range(4):
76     subset = df_p90[df_p90['Main_Cluster'] == subset_id].copy()
77
78     subset_X = StandardScaler().fit_transform(subset[numerical_features + ['Cmp%']])
79
```

```
    pca_subset = PCA(n_components=0.90, random_state=0)
    subset_X_pca = pca_subset.fit_transform(subset_X)

    inertia = []
    SI = []

    for k in range(2, 6):
        kmeans = KMeans(random_state=0, n_clusters=k)
        kmeans.fit(subset_X_pca)

        si = metrics.silhouette_score(subset_X_pca, kmeans.labels_)

        inertia.append(kmeans.inertia_)
        SI.append(si)

    print(f"\nMain-Cluster {subset_id}")
    plt.figure(figsize=(12,4))
    plt.subplot(1,2,1)
    plt.plot(np.arange(2,6),inertia)
    plt.xlabel('Number of clusters')
    plt.ylabel('Inertia')
    plt.xticks(np.arange(2,6))
    plt.subplot(1,2,2)
    plt.plot(np.arange(2,6),SI)
    plt.xlabel('Number of clusters')
    plt.ylabel('SI')
    plt.xticks(np.arange(2,6))
    plt.show()

subset_k = [3, 2, 4, 3]
tactical_roles = {}

for subset_id in range(4):
    subset = df_p90[df_p90['Main_Cluster'] == subset_id].copy()

    subset_X = StandardScaler().fit_transform(subset[numerical_features + ['Cmp%']])
    subset_X_pca = PCA(n_components=0.90, random_state=0).fit_transform(subset_X)

    subset_kmeans = KMeans(random_state=0, n_clusters=subset_k[subset_id])
    subset['Sub_Cluster'] = subset_kmeans.fit_predict(subset_X_pca)

    tactical_roles[f"Main_{subset_id}"] = subset

    print(f"Main-Cluster {subset_id}: {subset['Sub_Cluster'].nunique()} micro-roles, {len(subset)}
        players")


neighbors = NearestNeighbors(n_neighbors=5)
neighbors_fit = neighbors.fit(X_pca)
distances, indices = neighbors_fit.kneighbors(X_pca)

distances = np.sort(distances[:, 4], axis=0)

plt.figure(figsize=(8, 4))
plt.plot(distances)
plt.xlabel('Players (sorted by distance)')
plt.ylabel('5th Nearest Neighbor Distance (eps)')
plt.title('k-Distance Graph for DBSCAN')
plt.grid(True)
plt.show()

chosen_eps = 3.5
dbscan = DBSCAN(eps=chosen_eps, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_pca)

n_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
```

```
n_noise = list(dbscan_labels).count(-1)

print(f"Clusters: {n_clusters}")
print(f"Noise points: {n_noise}")

if n_clusters > 1:
    core_mask = dbscan_labels != -1
    si_dbscan = metrics.silhouette_score(X_pca[core_mask], dbscan_labels[core_mask])
    print(f"DBSCAN Silhouette Score: {si_dbscan:.3f}")
else:
    print("Found only 1 or 0 valid clusters.")

df_final = pd.concat(tactical_roles.values(), ignore_index=True)

df_final['Tactical_Role'] = 'M' + df_final['Main_Cluster'].astype(str) + '_S' + df_final['Sub_Cluster'
    ].astype(str)

print(f"Players categorized: {len(df_final)}")
print(f"Tactical Roles: {df_final['Tactical_Role'].nunique()}")

stat_names_mapping = {
    'G-PK': 'Non-Penalty Goals',
    'npxG': 'Non-Penalty Expected Goals',
    'Sh': 'Shots Total',
    'SoT': 'Shots on Target',
    'PrgR': 'Progressive Passes Received',
    'Ast': 'Assists',
    'xAG': 'Expected Assisted Goals',
    'KP': 'Key Passes',
    'SCA': 'Shot-Creating Actions',
    'PPA': 'Passes into Penalty Area',
    'PrgP': 'Progressive Passes',
    'Cmp%': 'Pass Completion %',
    '1/3': 'Passes into Final 3rd',
    'CrsPA': 'Crosses into Penalty Area',
    'Touches': 'Total Touches',
    'PrgC': 'Progressive Carries',
    'Carries': 'Total Carries',
    'Succ': 'Successful Take-Ons',
    'Dis': 'Dispossessed',
    'Recov': 'Ball Recoveries',
    'Tkl': 'Tackles Won',
    'Int': 'Interceptions',
    'Blocks_stats_defense': 'Blocks',
    'Clr': 'Clearances',
    'Att 3rd': 'Attacking 3rd Tackles',
    'Def 3rd_stats_possession': 'Touches (Defensive 3rd)',
    'Mid 3rd_stats_possession': 'Touches (Middle 3rd)',
    'Att 3rd_stats_possession': 'Touches (Attacking 3rd)',
    'Att Pen': 'Touches in Opp. Box',
    'Won': 'Aerial Duels Won'
}

role_means = df_final.groupby('Tactical_Role')[numerical_features].mean()
scaler = StandardScaler()
role_means_scaled = pd.DataFrame(scaler.fit_transform(role_means),
                                 columns=role_means.columns,
                                 index=role_means.index)

role_means_scaled = role_means_scaled.rename(columns=stat_names_mapping)

subset_scaled = role_means_scaled

fig, ax = plt.subplots(figsize=(16, 8))
cax = ax.imshow(subset_scaled, cmap='coolwarm', aspect='auto', vmin=-3, vmax=3)
fig.colorbar(cax, label='Z-Score (Standard Deviations from Mean)')
```

```
210
211 ax.set_xticks(np.arange(len(subset_scaled.columns)))
212 ax.set_yticks(np.arange(len(subset_scaled.index)))
213 ax.set_xticklabels(subset_scaled.columns, rotation=45, ha="right", fontsize=10)
214 ax.set_yticklabels(subset_scaled.index, fontsize=11)
215
216 ax.set_title('Tactical DNA: Statistical Strengths of Each Role (Red = High, Blue = Low)', pad=20,
        fontsize=14)
217 plt.tight_layout()
218 plt.show()
219
220 print(df_final['Tactical_Role'].value_counts().sort_index()) # cluster sizes
221 print(pd.crosstab(df_final['Tactical_Role'], df_final['Pos'])) # roles vs positions
222 for role, g in df_final.groupby('Tactical_Role'):
223     print(role, g.nlargest(5, 'Min')['Player'].tolist()) # example players
```